NLP with Transformers

Study Notes

Chapter 5: Text Generation
Chapter 6: Summarization (Introduction)

Exam-Oriented Revision Guide

Covers: Decoding Methods, Beam Search, Sampling, Summarization Pipelines

Compiled from *Natural Language Processing with Transformers* (O'Reilly)

Contents

Chapter 5: Text Generation

Overview and Motivation

Transformer-based language models can generate text that is nearly indistinguishable from human writing. GPT-2 (OpenAI) is a landmark example, demonstrating that a model trained purely to **predict the next word** on millions of web pages acquires broad pattern-recognition and task-transfer abilities.

GPT-2 and GPT-3 acquire skills such as arithmetic, translation, and unscrambling *without explicit supervision*. These tasks arise naturally in large pretraining corpora.

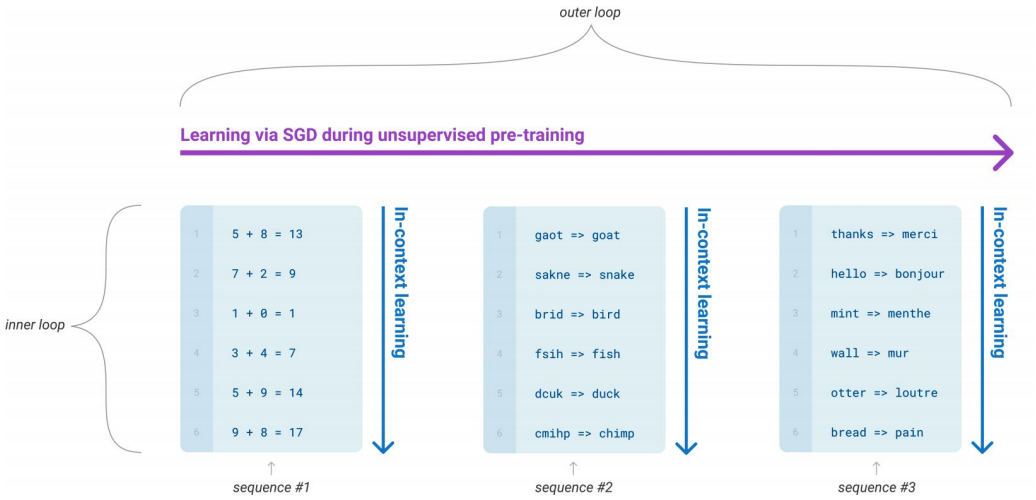

Figure 1: During pretraining, language models are exposed to sequences of tasks (arithmetic, unscrambling, translation) that can be adapted at inference time. (Courtesy of Tom B. Brown)

Figure 5-1 explained: The outer loop represents gradient updates via SGD during unsupervised pretraining. The inner loop shows *in-context learning*: the model sees several input–output examples in a sequence and infers the pattern for the final item, without any weight update. This mechanism is the foundation of few-shot prompting.

Applications enabled by text generation:

- Creative writing assistants (Write With Transformer, AI Dungeon)
- Conversational agents (Google Meena)
- Code generation, summarisation, and more

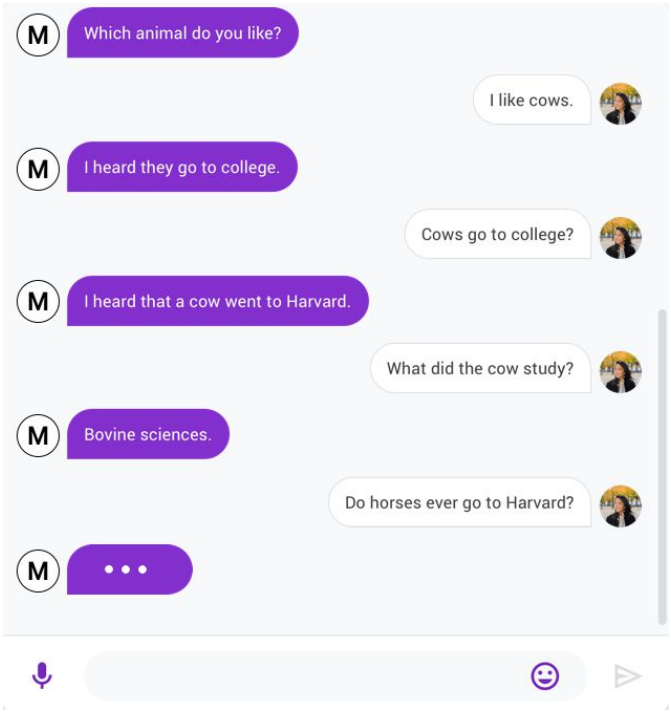

Figure 2: Google's Meena chatbot (left) telling a corny joke to a human (right). (Courtesy of Daniel Adiwardana and Thang Luong)

Figure 5-2 explained: Demonstrates that a language model can sustain multi-turn dialogue and generate contextually appropriate (even humorous) responses. Whether the model *intends* the joke is philosophically debated.

The Challenge with Generating Coherent Text

Unlike classification (single forward pass), text generation is:

- **Iterative** — one forward pass per generated token → high compute cost.
- **Sensitive to decoding strategy** — the choice of method and hyperparameters directly determines text quality and diversity.

A **decoding method** converts a model's continuous probability output (logits over vocabulary) into discrete tokens, one step at a time.

How Autoregressive / Causal Language Models Work

GPT-2 is a **causal (autoregressive) language model**. It estimates the probability of a token sequence $\mathbf{y} = y_1, y_2, \ldots, y_t$ given a context $\mathbf{x} = x_1, x_2, \ldots, x_k$:

$$P(\mathbf{y} \mid \mathbf{x}) = \prod_{t=1}^{N} P(y_t \mid y_{<t}, \mathbf{x}) \tag{1}$$

The joint probability is factorised as a *product of conditional probabilities* (Eq. 1). Each term is the probability of the next token given all previous tokens. This is practical to estimate; estimating the full joint distribution directly is not.

Contrast with BERT: BERT uses both past *and* future context (masked language modelling); GPT-2 uses only past context (causal masking).

Conditional Text Generation

- Start with a prompt (context $\mathbf{x}$).
- Predict next token, append it to input, repeat.
- Stop at a special `<EOS>` token or a maximum length.
- The output is **conditioned** on the input prompt → called **conditional text generation**.

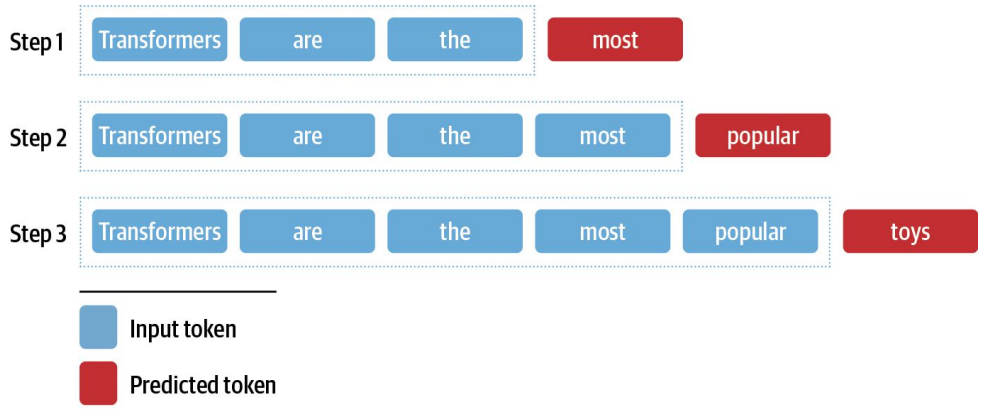

Figure 3: Generating text from an input sequence by appending the predicted token at each step.

Figure 5-3 explained: Each step (Step 1, 2, 3, ...) the model takes the growing sequence as input (blue boxes) and outputs the next predicted token (red box). The predicted token is appended before the next step. This iterative process is the core mechanism of all decoding methods.

Softmax over Vocabulary

The language model head outputs a logit $z_{t,i}$ for every vocabulary token w_i. The probability of the next token is:

$$P(y_t = w_i \mid y_{<t}, \mathbf{x}) = \text{softmax}(z_{t,i}) \tag{2}$$

Goal: Find the most likely overall sequence:

$$\hat{\mathbf{y}} = \arg\max_{\mathbf{y}} P(\mathbf{y} \mid \mathbf{x}) \tag{3}$$

This is intractable to compute exactly (exponential search space), so we use **approximations**.

Decoding Methods

1. Greedy Search Decoding

At each timestep, select the single token with the **highest probability**:

$$\hat{y}_t = \arg\max_{y_t} P(y_t \mid y_{<t}, \mathbf{x})$$

How it works (step-by-step)

1. Tokenise the input prompt.
2. Run a forward pass; collect logits for the *last* token position.
3. Apply softmax $\rightarrow$ probability distribution over vocabulary.
4. Select token with the maximum probability.
5. Append token to the sequence; repeat from step 2.

Listing 1: Greedy decoding (manual implementation)

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM

device = "cuda" if torch.cuda.is_available() else "cpu"
tokenizer = AutoTokenizer.from_pretrained("gpt2-xl")
model = AutoModelForCausalLM.from_pretrained("gpt2-xl").to(device
    )

input_ids = tokenizer("Transformers are the",
                      return_tensors="pt")["input_ids"].to(
                         device)

for _ in range(8):
    output = model(input_ids=input_ids)
    next_token_logits = output.logits[0, -1, :]
    next_token_probs  = torch.softmax(next_token_logits, dim=-1)
    next_token_id     = torch.argmax(next_token_probs)
    input_ids = torch.cat([input_ids, next_token_id[None, None]],
        dim=-1)
```

Example output

Starting from *"Transformers are the"*, greedy decoding produces:

> *"Transformers are the most popular toy line in the world,"*

This reveals that GPT-2 has internalised knowledge of the *Transformers* media franchise (created by Hasbro and Takara Tomy).

Listing 2: Using the built-in generate() for greedy decoding

```
output = model.generate(input_ids, max_new_tokens=8, do_sample=
    False)
print(tokenizer.decode(output[0]))
```

Drawbacks of Greedy Search

- Produces **repetitive output** — especially problematic for long text.
- **Misses higher-probability sequences** where the globally optimal path passes through locally low-probability tokens.
- Example with the unicorn prompt: "The researchers were surprised to find that the unicorns were able" — repeated verbatim.

Useful for **short, deterministic outputs** (e.g. arithmetic, factual Q&A) where correctness matters more than diversity. Example prompt: `"5 + 8 => 13 \n 7 + 2 => 9 \n 1 + 0 =>"`

2. Beam Search Decoding

Instead of keeping only the top-1 token, maintain the top-b **candidate sequences** (beams / partial hypotheses) at every step. At each timestep, expand all b beams by every possible next token, then keep only the b most probable extended sequences.

- b = **number of beams** (hyperparameter).
- Process repeats until `<EOS>` or max length is reached.
- Final answer = beam with highest **log-probability**.
- More beams → potentially better quality, but $b\times$ slower.

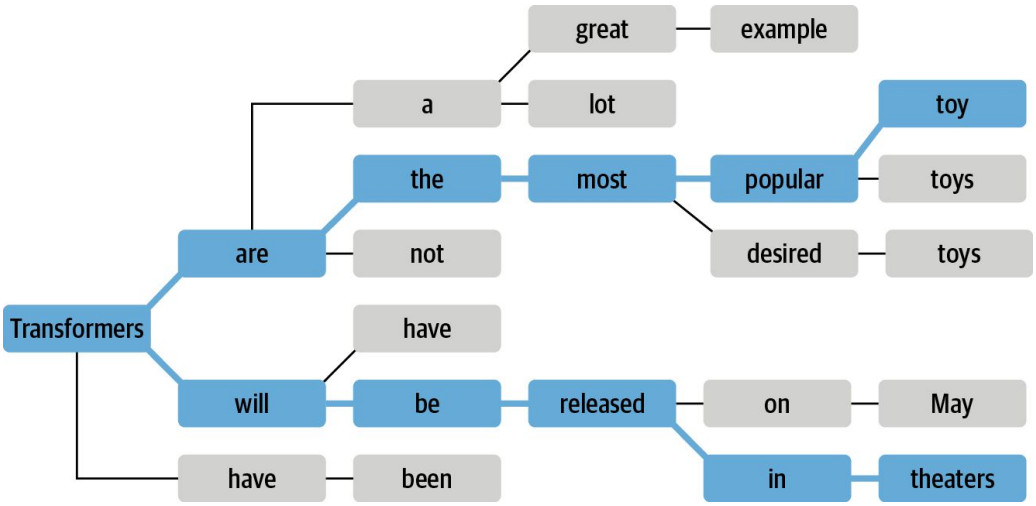

Figure 4: Beam search with two beams. The highlighted paths show the two candidate sequences tracked simultaneously.

Figure 5-4 explained: Starting from "Transformers", two beams are maintained at each step (shown in blue). Both candidate paths are expanded at every node, and only the two most probable are kept, pruning the rest. The beam that accumulates the highest total log-probability is selected at the end.

Why Log-Probability instead of Probability?

Joint probability involves a *product* of conditional probabilities (each $\in [0, 1]$):

$$P(y_1, \ldots, y_t \mid \mathbf{x}) = \prod_{t=1}^{N} P(y_t \mid y_{<t}, \mathbf{x})$$

For long sequences this **underflows** to zero (e.g. $0.5^{1024} \approx 5.6 \times 10^{-309}$).
Solution — apply logarithm:

$$\log P(y_1, \ldots, y_t \mid \mathbf{x}) = \sum_{t=1}^{N} \log P(y_t \mid y_{<t}, \mathbf{x})$$

Product $\longrightarrow$ Sum: numerically stable, easily comparable.

Listing 3: Computing sequence log-probability

```
import torch.nn.functional as F

def log_probs_from_logits(logits, labels):
    logp = F.log_softmax(logits, dim=-1)
    return torch.gather(logp, 2, labels.unsqueeze(2)).squeeze(-1)

def sequence_logprob(model, labels, input_len=0):
    with torch.no_grad():
        output = model(labels)
```

```
        log_probs = log_probs_from_logits(
            output.logits[:, :-1, :], labels[:, 1:])
        seq_log_prob = torch.sum(log_probs[:, input_len:])
    return seq_log_prob.cpu().numpy()
```

Beam Search in Practice

Listing 4: Beam search with 5 beams

```
output_beam = model.generate(input_ids, max_length=128,
                             num_beams=5, do_sample=False)
```

- Beam search (5 beams) achieved log-prob = **−55.23** vs greedy −87.43 (higher is better).
- However, beam search *still suffers from repetition.*

N-gram Penalty (no_repeat_ngram_size)

Track all n-grams already generated. Set the probability of any token that would create a *previously seen* n-gram to **zero**.

Listing 5: Beam search with n-gram penalty

```
output_beam = model.generate(input_ids, max_length=128,
                             num_beams=5, do_sample=False,
                             no_repeat_ngram_size=2)
```

- Log-prob drops to −93.12 (lower), but text becomes more **coherent and varied**.
- This trade-off is acceptable for tasks like **summarisation** and **machine translation** where factual accuracy matters.

Chapter Summary

Beam Search Summary

- Tracks b best sequences simultaneously → better than greedy.
- Uses log-probabilities for numerical stability.
- Still prone to repetition → mitigated by n-gram penalty.
- Best for tasks needing *factual accuracy* (translation, summarisation).

3. Sampling Methods

Basic (Random) Sampling

Instead of taking the argmax, **randomly sample** from the full vocabulary distribution:

$$P(y_t = w_i \mid y_{<t}, \mathbf{x}) = \text{softmax}(z_{t,i}) = \frac{\exp(z_{t,i})}{\sum_{j=1}^{V} \exp(z_{t,j})} \tag{4}$$

- Introduces diversity — different runs yield different outputs.
- Rare tokens may occasionally be selected → incoherence at higher temperatures.

Temperature Scaling

A **temperature parameter** T rescales the logits before the softmax:

$$P(y_t = w_i \mid y_{<t}, \mathbf{x}) = \frac{\exp(z_{t,i}/T)}{\sum_{j=1}^{V} \exp(z_{t,j}/T)} \tag{5}$$

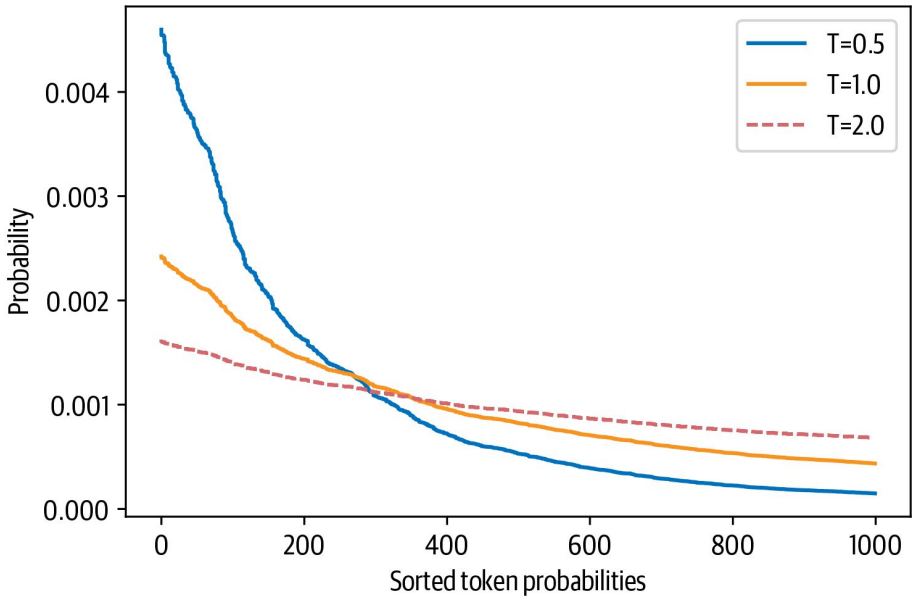

Figure 5: Distribution of randomly generated token probabilities for three temperatures ($T = 0.5$, $T = 1.0$, $T = 2.0$).

Figure 5-5 explained: Three curves show how temperature reshapes the probability distribution.

- $T \ll 1$ **(e.g. 0.5):** Distribution peaks sharply — top tokens dominate, rare tokens suppressed. Output: coherent but less creative.
- $T = 1.0$**:** Unmodified softmax.
- $T \gg 1$ **(e.g. 2.0):** Distribution flattens — all tokens become roughly equally likely. Output: diverse but often *gibberish*.

Temperature	Coherence	Diversity
Low ($T < 1$)	High	Low
$T = 1$	Moderate	Moderate
High ($T > 1$)	Low	High

Listing 6: Sampling with temperature

```
# High temperature (T=2): diverse but incoherent
output_temp = model.generate(input_ids, max_length=128,
                             do_sample=True, temperature=2.0,
                                top_k=0)
# Low temperature (T=0.5): coherent and readable
output_temp = model.generate(input_ids, max_length=128,
                             do_sample=True, temperature=0.5,
                                top_k=0)
```

Top-k and Nucleus (Top-p) Sampling

Both methods **truncate the vocabulary distribution** to avoid sampling very unlikely tokens, while still allowing diversity.

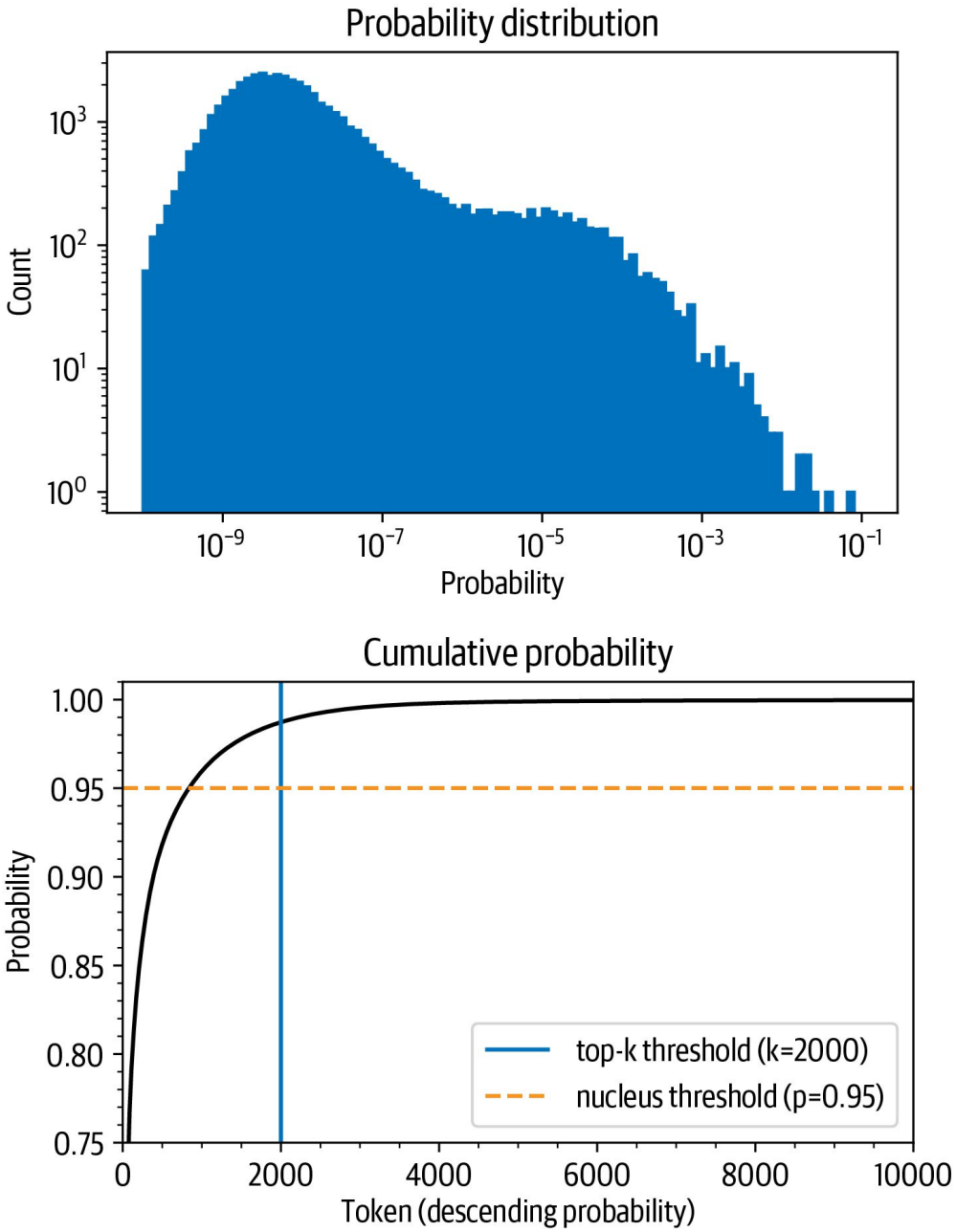

Figure 6: Probability distribution of next-token prediction (upper) and cumulative distribution of descending token probabilities (lower), showing top-k ($k = 2000$) and nucleus ($p = 0.95$) thresholds.

Figure 5-6 explained:

- **Upper plot:** Histogram of token probabilities — most tokens have very low probability; only a handful are reasonably likely.
- **Lower plot:** Cumulative probability. 96% of probability mass is concentrated in the top 1,000 tokens. The *vertical blue line* marks the top-k cut-off ($k = 2000$); the *horizontal orange dashed line* marks the nucleus threshold ($p = 0.95$).
- Sampling from the long tail even occasionally (1 in 100 chance per token) *compounds* over many tokens and degrades text quality.

Top-k Sampling

Restrict sampling to the k tokens with the **highest probability**. Everything outside the top-k is zeroed out (probability set to 0). This is equivalent to cutting at a *fixed vertical threshold* in the cumulative probability plot.

Listing 7: Top-k sampling ($k = 50$)

```
output_topk = model.generate(input_ids, max_length=128,
                             do_sample=True, top_k=50)
```

- Produces the most human-like output among sampling methods.
- **Limitation:** k is fixed regardless of the actual distribution shape — sometimes too broad, sometimes too narrow.

Nucleus (Top-p) Sampling

Instead of a fixed count, set a **probability mass threshold** p. Sort tokens by descending probability and include them until their cumulative probability $\geq p$. Sample only from this dynamic set.

- When the distribution is *peaked* $\rightarrow$ small nucleus (few tokens).
- When the distribution is *flat* $\rightarrow$ large nucleus (many tokens).
- Adapts to context automatically — more principled than top-k.

Listing 8: Top-p (nucleus) sampling ($p = 0.90$)

```
output_topp = model.generate(input_ids, max_length=128,
                             do_sample=True, top_p=0.90)
```

Combining Top-k and Top-p Both can be used together for best results:

Listing 9: Combined top-k + top-p

```
output = model.generate(input_ids, max_length=128,
                        do_sample=True, top_k=50, top_p=0.9)
# Selects from at most 50 tokens with cumulative prob mass of 90%
```

Beam search can also be combined with sampling: instead of greedily selecting the top-b tokens for the next beam, *sample* them, then build beams normally.

Which Decoding Method Is Best?

Task Type	Recommended Method	Reason
Arithmetic / factual Q&A	Greedy or beam search	Deterministic, correct
Summarisation / translation	Beam search + n-gram penalty	Accurate, less repetitive
Creative writing / chat	Sampling (top-k / top-p)	Diverse, human-like
Long open-domain text	Top-k + nucleus + temperature	Balance creativity & coherence

The optimal decoding strategy depends on the task. For precision tasks, use deterministic methods. For creative or diverse tasks, use sampling. Always tune hyperparameters (temperature, k, p, num_beams) empirically.

Chapter 5 Conclusion

Chapter Summary

- Text generation requires **one forward pass per token** $\rightarrow$ expensive.
- **Greedy search**: fast, simple, repetitive.
- **Beam search**: better global probability, still repetitive; fix with n-gram penalty.
- **Temperature**: controls coherence vs. diversity trade-off.
- **Top-k sampling**: fixed vocabulary cut-off.
- **Top-p (nucleus) sampling**: dynamic cut-off based on probability mass.
- Evaluation of generated text requires task-specific metrics (covered in Chapter 6).

Chapter 6: Summarisation (Introduction)

Why Summarisation is Hard

Text summarisation is a **sequence-to-sequence (seq2seq)** task requiring:

- Understanding of long passages.
- Reasoning about content.
- Generation of fluent, faithful output.
- Domain generalisation (news ≠ legal contracts).

- **Extractive:** Copies sentences directly from the source text.
- **Abstractive:** Generates *new sentences* capturing the main ideas (more human-like, more challenging).

Summarisation is best suited to **encoder-decoder** transformer architectures (e.g. T5, BART, PEGASUS) because:

- The encoder reads and understands the long input.
- The decoder generates the shorter output summary.

The CNN/DailyMail Dataset

- ∼**300,000** article–summary pairs from CNN and DailyMail.
- Summaries are the *bullet-point highlights* attached to each article.
- Summaries are **abstractive** (new sentences, not excerpts).
- Version 3.0.0 used (non-anonymised, for summarisation).
- Columns: `article`, `highlights`, `id`.

Listing 10: Loading the CNN/DailyMail dataset

```
from datasets import load_dataset
dataset = load_dataset("cnn_dailymail", version="3.0.0")
print(dataset['train'].column_names)
# ['article', 'highlights', 'id']
```

Key challenge: Articles can be **17× longer** than their summaries. Most transformer models have a context limit of ∼1,000 tokens → articles must be **truncated**, potentially losing important information.

Text Summarisation Pipelines

Sentence Tokenisation with NLTK

A convention is to separate summary sentences with newlines. NLTK's `sent_tokenize()` handles abbreviations (e.g. "U.S.", "U.N.") correctly:

Listing 11: Sentence tokenisation with NLTK

```
import nltk
from nltk.tokenize import sent_tokenize
nltk.download("punkt")

string = "The U.S. are a country. The U.N. is an organization."
sent_tokenize(string)
# ['The U.S. are a country.', 'The U.N. is an organization.']
```

Baseline: First Three Sentences

Listing 12: Simple extractive baseline

```
def three_sentence_summary(text):
    return "\n".join(sent_tokenize(text)[:3])
```

A surprisingly competitive baseline for news articles (news articles typically front-load key information).

Surveyed Summarisation Models

GPT-2 (Zero-shot with TL;DR Prompt)

- GPT-2 is a **decoder-only** model, not designed for summarisation.
- Zero-shot trick: append `"\nTL;DR:\n"` to the article.
- GPT-2 has seen this pattern on Reddit during pretraining.

Listing 13: GPT-2 summarisation via TL;DR

```
from transformers import pipeline, set_seed
set_seed(42)
pipe = pipeline("text-generation", model="gpt2-xl")
gpt2_query = sample_text + "\nTL;DR:\n"
pipe_out = pipe(gpt2_query, max_length=512,
                clean_up_tokenization_spaces=True)
```

T5 (Fine-tuned Text-to-Text)

T5 (Text-To-Text Transfer Transformer) reformulates *every NLP task* as a text-to-text problem. Summarisation input: `"summarize: <article>"`. Training mixes unsupervised (masked language modelling) and supervised tasks.

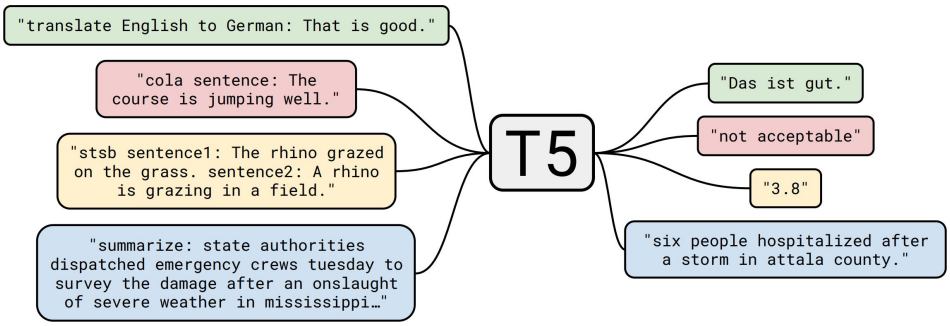

Figure 7: T5's text-to-text framework. A single model handles translation, summarisation, linguistic acceptability (CoLA), and semantic similarity (STSB) by using task-specific text prefixes. (Courtesy of Colin Raffel)

Figure 6-1 explained: By casting all tasks as text-in → text-out, T5 can be fine-tuned on any task using the same architecture. The prefix in the input (e.g. `"summarize:"`) signals which task to perform. This extreme versatility makes T5 a powerful baseline.

Listing 14: T5 summarisation pipeline

```
pipe = pipeline("summarization", model="t5-large")
pipe_out = pipe(sample_text)
summaries["t5"] = "\n".join(sent_tokenize(pipe_out[0]["
    summary_text"]))
```

BART

- **Encoder-decoder** architecture.
- Trained to **reconstruct corrupted (noised) inputs** (combines BERT's masked LM with GPT's autoregressive generation).
- `facebook/bart-large-cnn`: specifically fine-tuned on CNN/DailyMail.

Listing 15: BART summarisation pipeline

```
pipe = pipeline("summarization", model="facebook/bart-large-cnn")
pipe_out = pipe(sample_text)
summaries["bart"] = "\n".join(sent_tokenize(pipe_out[0]["
    summary_text"]))
```

PEGASUS

PEGASUS (Pre-training with Extracted Gap-Sentences):

- Encoder-decoder transformer.
- Pre-training objective: *predict masked sentences* (gap-sentence generation).
- Sentences selected are those with the highest content overlap with surrounding paragraphs (measured by summarisation metrics).
- The pretraining objective closely mimics the downstream summarisation task → state-of-the-art on summarisation benchmarks.

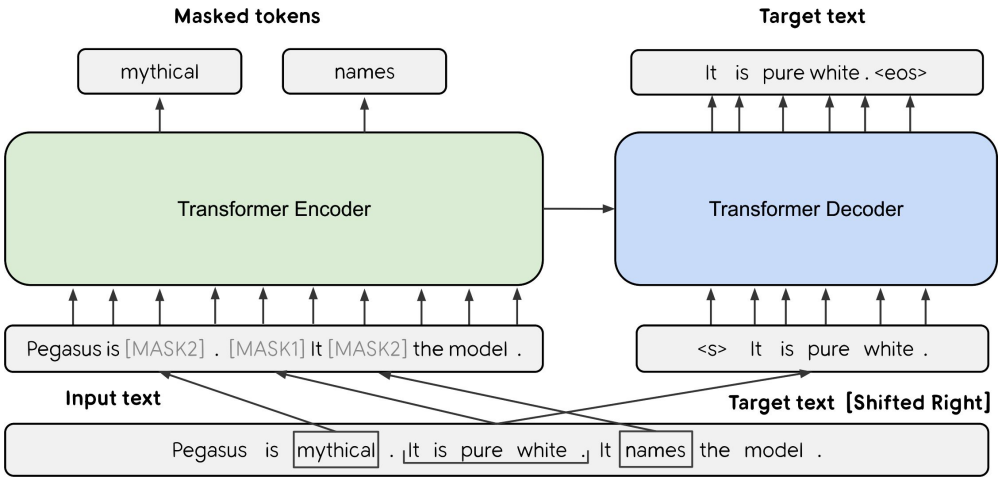

Figure 8: PEGASUS architecture. The model is pre-trained to reconstruct gap sentences (masked in the input), which closely mirrors the summarisation task. (Courtesy of Jingqing Zhang et al.)

Figure 6-2 explained: Selected sentences (those containing the most information about the document) are masked from the input and the model must reconstruct them as the target. Because this task is structurally similar to summarisation, PEGASUS transfers very effectively to downstream summarisation tasks with minimal fine-tuning.

Listing 16: PEGASUS summarisation pipeline

```
pipe = pipeline("summarization", model="google/pegasus-
    cnn_dailymail")
pipe_out = pipe(sample_text)
# PEGASUS uses a special <n> token for newlines
summaries["pegasus"] = pipe_out[0]["summary_text"].replace(" .<n>
    ", ".\n")
```

Model Comparison Summary

Model	Type	Training	Notes
GPT-2	Decoder-only	Not trained on summarisation	Zero-shot via TL;DR prompt
T5	Enc-Dec	Multi-task incl. summarisation	Text-to-text; no fine-tuning needed
BART	Enc-Dec	Pretrained + fine-tuned on CNN/DM	Denoising objective
PEGASUS	Enc-Dec	Pretrained + fine-tuned on CNN/DM	Gap-sentence objective

Chapter Summary

Chapter 6 Introduction Summary

- Summarisation is a seq2seq task — encoder-decoder models are natural fits.
- CNN/DailyMail provides 300k abstractive article-summary pairs.
- Context length limits (typically ∼1,000 tokens) require article truncation.

- GPT-2 (zero-shot), T5, BART, and PEGASUS all achieve reasonable results.
- PEGASUS achieves the best results by aligning pretraining with the target task.
- Evaluation requires automatic metrics (ROUGE etc.) — covered in detail later in Ch. 6.

Quick-Reference Formula Sheet

Autoregressive factorisation:

$$P(\mathbf{y} \mid \mathbf{x}) = \prod_{t=1}^{N} P(y_t \mid y_{<t}, \mathbf{x})$$

Greedy decoding:

$$\hat{y}_t = \arg\max_{y_t} P(y_t \mid y_{<t}, \mathbf{x})$$

Softmax (next-token probability):

$$P(y_t = w_i \mid y_{<t}, \mathbf{x}) = \frac{\exp(z_{t,i})}{\sum_{j=1}^{V} \exp(z_{t,j})}$$

Temperature-scaled softmax:

$$P(y_t = w_i \mid y_{<t}, \mathbf{x}) = \frac{\exp(z_{t,i}/T)}{\sum_{j=1}^{V} \exp(z_{t,j}/T)}$$

Log-probability of a sequence (beam search scoring):

$$\log P(y_1, \ldots, y_t \mid \mathbf{x}) = \sum_{t=1}^{N} \log P(y_t \mid y_{<t}, \mathbf{x})$$

Optimal sequence (goal):

$$\hat{\mathbf{y}} = \arg\max_{\mathbf{y}} P(\mathbf{y} \mid \mathbf{x})$$

- `do_sample=False` → deterministic (greedy / beam)
- `num_beams=`b → beam search with b beams
- `no_repeat_ngram_size=`n → n-gram penalty
- `temperature=`T → scale logits ($T < 1$: sharper, $T > 1$: flatter)
- `top_k=`k → sample from top-k tokens only
- `top_p=`p → sample from tokens summing to probability p
- `max_new_tokens=`n → limit generated length